Mohammad's Development Rules

Oct 31, 2026 · @Mohammad

These are the rules you keep asking agents to follow on your Laravel 13 + Livewire projects, written down once so every agent applies them without being told. The last section holds a short prompt to paste into a project's CLAUDE.md.

How to read this

Each rule carries a tag and a level:

Tag	Meaning
[H]	From your history: you asked for it in past sessions or it is already written into one of your repos
[+]	Added from well-established practice (Laravel docs, OWASP, WCAG, Spatie guidelines) to round out a rule you already follow
MUST	Never broken without your explicit approval
SHOULD	Default; deviate only with a stated reason

Where the [H] rules come from: about 100 Claude Code sessions (Oct 2026), about 220 pull requests on ResiCore and OrganiCore, and the existing rule files in GreenHumanResource (`CLAUDE.md`, the most complete), ResiCore (`CLAUDE.md`, `.ai/entity-management.md`), IMEXCO (`CLAUDE.md`, `ThrottlesSubmissions`), OrganiCore and your `laravel-translation-sync` package. Session transcripts were read by title and summary, not message by message, so a few one-off wishes may be missing.

1. Architecture and layering

Every request flows one way: route → controller or Livewire component → service → model, and the view only renders what it is handed.

Thin entry points

- **[H] MUST** Controllers and Livewire components contain no Eloquent queries (`::where`, `::find`, `::create`), no `DB::`, no `->save()/->update()/->delete()` and no business rules. They hold UI state, validate, authorize, delegate to a service and map the result back.

- **[H] MUST** All data access and business logic live in a `<Domain>Service` under `app/Services/` (sub-domains in sub-folders, e.g. `app/Services/Payroll/`). One service per aggregate, not one per table.
- **[H] MUST** Inject services: constructor or action type-hint in controllers; `boot()` into a `protected` property in Livewire (never a public, serialized property). Never `new`, `app()` or facades to reach models.
- **[H] MUST** Services return domain data (models, collections, paginators, DTOs, arrays), never views or HTTP responses.
- **[H] SHOULD** Cross-cutting concerns (notifications, activity log, PDF, exports, CRUD notifications) are their own services or listeners, composed by DI, not copied.
- **[H] SHOULD** Long-running work (bulk dues generation, imports, notifications to many users) goes to queued jobs.

Design patterns you use

Pattern	Where	Level
Service layer	`app/Services/*Service.php`	[H] MUST
Policies + Gates per entity, permission names `{entity}.{action}`	`app/Policies`, `ActionPermissions`	[H] MUST
Form objects / Form Requests for validation	Livewire Forms, `app/Http/Requests`	[+] SHOULD
Enums for statuses and types (TitleCase keys)	`app/Enums`	[H] SHOULD
Traits/Concerns for shared model behaviour (`EnforcesSoftDelete`, `MaintainsAuditChain`, `ThrottlesSubmissions`)	`app/Models/Concerns`, `app/Support`	[H] SHOULD
Observers / Events + queued Listeners for side effects	`app/Observers`, `app/Listeners`	[H] SHOULD
Reusable Blade components for repeated UI (flash message, hero, action buttons, sort chip)	`resources/views/components`	[H] MUST
DTOs (readonly classes) when a service needs more than 3–4 loose arguments	`app/Data`	[+] SHOULD
Strategy/driver interfaces for swappable providers (payment, AI provider via Prism)	`app/Contracts` + bindings	[+] SHOULD

Principles behind it

- **[+] SHOULD** SOLID, especially single responsibility and dependency inversion: depend on interfaces where a second implementation is realistic.
- **[+] SHOULD** Don't add a repository layer on top of services; Eloquent already is one. Match the altitude of the existing code.
- **[H] MUST** Reuse existing components, services and helpers before creating new ones; mirror the nearest well-factored sibling file.
- **[H] MUST** No new top-level folders and no dependency changes without approval.

2. Views and frontend

Blade renders view-ready data; styling lives in stylesheets and behaviour lives in Livewire, Alpine or bundled JS.

No inline PHP, CSS or JS

- **[H] MUST** No `@php … @endphp` blocks and no logic in Blade. `@if`, `@foreach`, `@forelse`, `@can` for rendering are fine; computing, filtering, grouping, formatting (dates, money, percentages) belongs in the service or component.
- **[H] MUST** No `style="…"` attributes and no `<style>` blocks. Use Tailwind utilities or the project stylesheet in `resources/css/` (e.g. `resicore.css`, `imexco-admin.css`). Only exceptions: a truly data-driven value (`style="width: {{ $pct }}%"`, preferably via a CSS custom property) and DomPDF templates in `resources/views/pdf/`.
- **[H] MUST** No `<script>` tags and no native `onclick=` / `onchange=` handlers in Blade. Order of preference: Livewire `wire:*` → Alpine (`x-data`, `@click`, `x-on:change`) → a module in `resources/js/` bundled by Vite.
- **[+] SHOULD** Keep the app compatible with a strict Content-Security-Policy (no `unsafe-inline`), which these three rules make possible.

Layout and components

- **[H] MUST** No `<table>` for admin and list screens. Render records as responsive cards (card grid, item card with head / `dl` meta / actions) with sort chips and a filter bar. A table is acceptable only for genuinely tabular reports (balances, BI) and must stack on phones.
- **[H] MUST** One shared design system per project (classes prefixed per project, e.g. `rc-`, `ix-`); reuse its components (flash message, page hero, CRUD action buttons, sort chip) instead of one-off markup.
- **[H] SHOULD** Keep screens simple: at most two sections per component; split anything bigger into its own page.
- **[H] SHOULD** Navigation is grouped by role/area (e.g. Habitat, Syndic team, Platform admin), with breadcrumbs, a collapsible sidebar and the active item highlighted; language switcher and theme toggle sit in the top bar.
- **[H] SHOULD** Icons and logos are sized by CSS classes, consistently per heading level.
- **[+] MUST** Accessibility to WCAG 2.2 AA: labels on every input, visible focus, keyboard-reachable menus, colour contrast ⩾ 4.5:1, `dir="rtl"` honoured for Arabic.
- **[+] SHOULD** Mobile first: every screen is checked at phone width; no horizontal scroll.
- **[H] MUST** After any CSS/JS change, remind that `npm run build` is needed to see it.

3. Routes and screens

Each task gets its own URL: you have repeatedly asked to pull forms, tabs and detail panels out of list pages into dedicated routes.

- **[H] MUST** Create and edit forms are their own full-page Livewire components on their own routes (`/budgets/create` , `/budgets/{budget}/edit`), not modals or inline panels on the list page.
- **[H] MUST** Tabs that are really separate reports or areas become separate routes (BI report per category, visitor announcement, demo page, FAQ page, terms page, notifications "see all" page).
- **[H] MUST** URL segments and route names are in English and kebab-case (`my-lot` , `dues`), never French or Arabic, whatever the UI language.
- **[H] MUST** Use named routes and `route()` everywhere; group admin routes by feature and protect each group with permission middleware.
- **[H] SHOULD** Filters and periods (month, quarter, year) persist in the query string so a screen can be bookmarked and shared.
- **[H] SHOULD** Long lists are paginated (25 per page by default), including audit logs.
- **[+] SHOULD** RESTful naming (`index` , `create` , `store` , `show` , `edit` , `update` , `destroy`); APIs versioned (`/api/v1`) with Eloquent API Resources and JSON errors.

4. Translations and i18n

No user-facing string is ever hard-coded; every string is a key in a per-feature language file, present in every locale.

- **[H] MUST** Every visible string (labels, buttons, menus, flash messages, validation and email text, seeder content shown to users) goes through `__()` / `trans_choice()` .
- **[H] MUST** Keys are grouped into one bundle per feature or screen: `lang/{locale}/{feature}.php` (`billing.php` , `faq.php` , `nav.php`), nested by dots (`settings.admin.fields.temperature`). No catch-all file for new keys, no JSON string-as-key files for new work.
- **[H] MUST** Every key exists in every supported locale: en, fr, ar on ResiCore and IMEXCO; en, fr, ar, de, es on OrganiCore. Run `php artisan translations:sync` (your `laravel-translation-sync` package) after adding keys, then fill every `[TODO]` .
- **[H] SHOULD** Add a test that fails on missing translations for new menu entries and pages.
- **[H] SHOULD** Translatable database content (CMS pages, FAQ, plans) is stored per locale (e.g. `spatie/laravel-translatable`) and seeded in all languages, Arabic included.

- **[H] SHOULD** The chosen locale persists through login and is switchable from the top bar in both the public site and admin.
- **[+] MUST** Arabic renders right-to-left: `dir` and `lang` set on `<html>`, logical CSS properties (`ms-` / `me-` , `start` / `end`) instead of `left` / `right` .
- **[+] SHOULD** Use placeholders (`:name`) and pluralisation instead of concatenating strings; format dates, numbers and currency with the locale.

5. Security

Every public entry point is rate-limited, every action is authorized, and money is never hard-deleted.

Rate limiting

- **[H] MUST** Every public (front-office) form and endpoint is rate-limited: contact, demo request, newsletter, quote request, search, login, password reset, verification links, API auth, payment webhooks.
- **[H] MUST** Named limiters are declared with `RateLimiter::for()` in a service provider and applied with `throttle:<name>` middleware; Livewire forms use a shared trait (IMEXCO's `ThrottlesSubmissions`) that counts every attempt, valid or not, and returns a translated validation error.
- **[H] MUST** Limits are configurable (e.g. `config/site.php` → `form_throttle.<form>`), keyed by form + IP (+ email where relevant). Reference values from your code: auth 6/min, API 60/min, webhooks 60/min, public forms 10/min.
- **[H] SHOULD** Public forms add a honeypot field; AI assistant endpoints have per-user limits and an audit log.

Authorization and data protection

- **[H] MUST** Every Create/Read/Update/Delete goes through a Policy; permissions follow `{entity}.{action}` (`expenses.create` , `payments.refund`). Hide buttons with `@can` , and authorize again on the server.
- **[H] MUST** Users see only their own tenant/building/space; role areas (platform admin, syndic, resident) are separated, with a space chooser at login.
- **[H] MUST** Financial records (payments, expenses, incomes, budgets, salaries, invoices) use soft deletes only; `forceDelete` is forbidden. Prefer soft deletes on every model.
- **[H] MUST** Changes are logged with `spatie/laravel-activitylog` (who, what, old → new) so the audit trail stays complete.
- **[H] SHOULD** Consent tracking and personal-data export exist where personal data is stored (GDPR).

- **[+] MUST** Validate all input server-side; mass assignment only through `$fillable` or validated arrays; escape output with `{{ }}`, sanitise any HTML you must render with `{!! !!}`.
- **[+] MUST** Keep CSRF on (and keep the token valid across multi-step logins); no secrets in code or seeders; `.env` never committed.
- **[+] SHOULD** Security headers (CSP, HSTS, X-Content-Type-Options, Referrer-Policy) and signed URLs for one-off links; follow the OWASP Top 10.

6. Data and performance

Schema changes go through migrations, queries are indexed and eager-loaded, and heavy work runs in the queue.

- **[H] MUST** Every schema or data change is a migration (including one-off data fixes such as repricing); never edit the database by hand. Mention `php artisan migrate` in the hand-off.
- **[H] MUST** Size columns for real data (the quote `reference` overflow); money as integer minor units or `decimal`, never float.
- **[H] SHOULD** Index foreign keys and every column used in `where`, `orderBy` or status filters.
- **[H] SHOULD** Seeders come in tiers (minimal / demo / full) and seed realistic data for every building or tenant, in all locales.
- **[H] SHOULD** Queue long-running work; queued listeners for notifications.
- **[+] MUST** No N+1 queries: eager-load relations (`with()`), enable `Model::preventLazyLoading()` outside production.
- **[+] SHOULD** Cache expensive read-only aggregates (dashboards, feature flags) and invalidate on write; read settings once per request.
- **[+] SHOULD** Wrap multi-step writes in `DB::transaction()` inside the service.

7. Code quality and testing

A change is done only when format, static analysis and the relevant tests pass.

- **[H] MUST** PHP 8.4 style: `strict` types, explicit return types and parameter types, constructor property promotion, curly braces on every control structure, generic PHPDoc for collections (`Collection<int, User>`), `@property` annotations on models and components.
- **[H] MUST** Descriptive names (`isRegisteredForDiscounts`, not `discount()`); PHPDoc only where logic is not obvious.

- **[H] MUST** Run `vendor/bin/pint --dirty` and PHPStan/Larastan (level 5 or the project's level) before committing; fix new errors instead of growing the baseline.
- **[H] MUST** Pest tests (feature tests by default, factories over manual setup) for every behaviour or logic change, covering failure paths and authorization. Skip tests for pure copy or styling changes.
- **[H] MUST** Never delete or skip a test to get green; fix the code or update an out-of-date expectation and say so.
- **[H] SHOULD** Test runs write JUnit and TestDox reports to `storage/test-reports/`; CI uploads them.
- **[H] MUST** Be honest in the hand-off: say exactly which tests, Pint and PHPStan ran, and which could not run and why.
- **[H] SHOULD** Don't create documentation files unless asked; when docs exist, keep the generated codebase reference current.
- **[+] SHOULD** Small functions, early returns, no dead code, no commented-out code.

8. Git workflow and delivery

All work lands on `develop` through a pull request; `main` only receives releases.

- **[H] MUST** Branch from `develop` (`feature/`, `bugfix/`, `refactor/`, `docs/`, `chore/`) and open the PR against `develop`, never `main`.
- **[H] MUST** Conventional commit messages: `type(scope): subject` (`feat`, `fix`, `refactor`, `docs`, `chore`, `test`).
- **[H] MUST** No force-push to `develop` or `main`; no direct pushes to `main`.
- **[H] SHOULD** One concern per PR; merge `develop` into the branch and resolve conflicts before review, keeping `develop`'s side unless asked otherwise.
- **[H] SHOULD** PR descriptions list what changed, migrations to run, build steps (`npm run build`) and what was and wasn't tested, and stay accurate.
- **[+] SHOULD** Keep PRs reviewable (roughly under 400 changed lines where possible).

9. Agent prompt (short version)

Paste this into each project's `CLAUDE.md` (or `AGENTS.md`). It is also saved as a file, `coding-rules/CLAUDE.md`, in the project files. It keeps only the MUST rules and ends with grep guards an agent runs before handing back.

```
# House rules (Mohammad) — read before every change

Stack: Laravel 13, Livewire 3/4, Alpine, Tailwind, Pest, PHP 8.4. These rules beat your defaults.
If a rule conflicts with the task, stop and ask. Check every item in "Before you finish".

## Architecture
1. Controllers and Livewire components only hold state, validate, authorize and delegate.
   No Eloquent queries, `DB::`, `->save()/update()/delete()` or business logic in them.
2. All data access and business logic live in `app/Services/<Domain>Service.php`.
   Inject services (constructor in controllers, `boot()` into a `protected` property in Livewire).
   Services return data, never views or responses.
3. Use the project's patterns: Policies per entity (`{entity}.{action}` permissions), Enums for
   statuses, Concerns/traits for shared behaviour, Observers/queued Listeners for side effects,
   queued Jobs for long work. Reuse existing components and services before writing new ones.
4. No new top-level folders and no dependency changes without approval.

## Views
5. No `@php` blocks or logic in Blade. Pass view-ready (already formatted) data.
6. No `style="…"` and no `<style>`. Use Tailwind or the project stylesheet in `resources/css/`.
   Exceptions: data-driven values (`width: {{ $pct }}%`) and `resources/views/pdf/`.
7. No `<script>` and no `onclick=`/`onchange=`. Use `wire:*`, then Alpine, then `resources/js/` modules.
8. No `<table>` for lists or admin screens: responsive cards with sort chips and a filter bar.
9. Reuse the design-system components (flash, hero, action buttons). Max two sections per
   component; mobile-first; WCAG 2.2 AA; RTL works for Arabic.

## Routes
10. Create/edit forms, report tabs and detail panels get their own route and full-page component.
11. URLs and route names are English kebab-case (`my-lot`, `dues`). Always `route('name')`.
12. Paginate lists (25). Persist filters in the query string.
```

```
## Translations
13. Every visible string uses `__()`. Keys live in a per-feature bundle
`lang/{locale}/{feature}.php`
    with nested dot keys. Never hard-code text.
14. Add every key to every locale in `lang/` (en, fr, ar; + de, es where
present).
    Run `php artisan translations:sync` and replace every `[TODO]`.

## Security
15. Rate-limit every public form and endpoint (login, reset, contact, demo,
quote, search,
    newsletter, API auth, webhooks): `RateLimiter::for()` + `throttle:
<name>`, or the
    `ThrottlesSubmissions` trait for Livewire. Limits come from config, keyed
by form + IP.
16. Authorize every CRUD action through a Policy, on the server, and with
`@can` in the view.
    Scope data to the current tenant/building/space.
17. Financial models use soft deletes only; never `forceDelete`. Log changes
with activitylog.
18. Validate all input server-side; no secrets in code; keep CSRF on.

## Data
19. Every schema or data change is a migration. Index filtered and foreign-
key columns.
    Eager-load relations (no N+1). Money is never a float. Multi-step writes
use `DB::transaction()`.

## Quality and Git
20. Strict types, return types, constructor promotion, curly braces, generic
PHPDoc.
21. Pest tests for every behaviour change, including failure and
authorization paths.
    Never delete or skip a test to get green.
22. Run `vendor/bin/pint --dirty`, PHPStan and the narrowest relevant tests.
23. Branch from `develop`, PR into `develop` (never `main`), commits as
`type(scope): subject`.
    No force-push to `develop`/`main`.

## Before you finish (self-check)
```bash
grep -rn '@php' resources/views                                    # must be
empty
grep -rn '<script' resources/views                                 # must be
empty
```
```

```
grep -rn '<style' resources/views --exclude-dir=pdf                # must be
empty
grep -rnE '\son(click|change|submit|input|keydown)=' resources/views  # must
be empty
grep -rn '<table' resources/views --exclude-dir=pdf                # lists
must use cards
grep -rnE '::(where|find|create|all|first)\(|DB::' app/Livewire
app/Http/Controllers  # must be empty
php artisan translations:sync --dry-run                            # no
missing keys
```
- [ ] Logic in a service; entry points thin
- [ ] No inline PHP/CSS/JS; cards not tables
- [ ] Own route per form/report; English URLs
- [ ] All strings translated in every locale
- [ ] Public endpoints rate-limited; actions authorized
- [ ] Migration + tests added; Pint, PHPStan, tests run
- [ ] Hand-off states exactly what ran, what did not, and any `migrate` / `npm run build` needed

Sources

- Your repos: GreenHumanResource `CLAUDE.md`, ResiCore `CLAUDE.md` and `.ai/entity-management.md`, IMEXCO `CLAUDE.md` and `app/Support/Livewire/ThrottlesSubmissions.php`, OrganiCore `CLAUDE.md` and `routes/`, laravel-translation-sync
- Pull request history on ResiCore (#120–#219) and OrganiCore (#533–#632); Claude Code session titles and summaries, Oct 2026
- Added practice: Laravel rate limiting, OWASP Top 10, OWASP CSP cheat sheet, WCAG 2.2, Spatie Laravel guidelines, Conventional Commits